Andre Uche

New Jersey, United States | andreuchennaa@gmail.com | 704-412-7917

Profile

I have never really seen data as just numbers sitting in storage or abstract intangible concepts. For me, data is more than that, data is human, data is alive. It is a moving thing, always changing, always talking back to you if you know how to listen. That is why I love building systems that create feedback loops on top of feedback loops. When I design a system, I want it to be more than a one-way street. I want it to catch its own mistakes, learn from its own patterns, and get sharper every single day. That is the fun part for me. For the past nine years I have been living in this space across mostly AWS, Azure, and a bit of GCP, chasing hard problems and viewing those problems as opportunities. I enjoy breaking down a big, stubborn issue into smaller parts, solving each one carefully, and then watching the whole machine come together. I keep the users at the center of everything I build, because at the end of the day it is about trust. When data works the way it should, traders can make decisions with confidence, regulators can rely on reports without second-guessing, and we can look at a dashboard and believe what they see. I think of myself as both an engineer and a gardener. I plant structures where they need to be, I prune what is slowing the system down, and I nurture the parts that are growing well so they can flourish. I use tools like Python, Go, Databricks, ClickHouse and Airflow, but the tools are not the point. The tool could change but the obejctive is always the same. The point is to build something that keeps improving itself and teaching me in the process. That is what keeps me hooked, that is why I still love this work.

Professional Experience

Senior Data Platform Engineer, Gemini Trust Company, LLC, Remote

Jul 2022 — Present

- At Gemini the analytics stack was always moving. Data was constantly flowing in from AWS DMS into S3. From there, Databricks picked it up and layered it into bronze, silver, and gold tables, making sure masking was in place where needed, and then pushed the curated outputs into analytics and dashboards.
- Snowflake was still active during the transition, but the plan was to gradually move away from it and go fully lake-first. Airflow itself was spread across multiple AWS accounts, so one DAG might exist in more than one place. With that kind of moving target, my job was to keep the whole path dependable, even while new tables, new events, and new DAGs were being introduced almost every week.

Schemas treated like code

- Before I stepped in, schema management was just too loose. Engineers would jump into Databricks or Snowflake and add a column directly by hand. It solved their problem in the moment, but the bigger issue was that staging and production quickly went out of sync. Dashboards would quietly misbehave, and compliance teams could even end up using mismatched fields.
- I fixed it by insisting that schema changes be treated like proper code. Every new column, mask, grant, or description was stored in Git. I built a pipeline that rolled these changes out the way software is deployed: first into the lake, validate, then into analytics. Metadata like grants and comments travelled with the table. Now reviewers saw the real diffs in pull requests, not screenshots, and those quiet edits stopped completely. This gave Gemini consistency across environments, which was very important for regulatory reporting.

Stabilizing DMS to S3 to Databricks autoload

- A serious hidden issue was that DMS would sometimes lag, or the format of the landing files in S3 would change slightly. Dashboards would still show everything as green, but in reality the lake was hours behind. That is very risky when you are dealing with trading balances and compliance data that must always be fresh.
- I added checks right at the landing path. We monitored whether new files were landing on schedule, whether partitions were moving forward, and whether record counts were shifting the way they should. If anything was off, the job failed immediately with a reason logged. That way, we avoided pushing stale data downstream.
- Another challenge was that source systems were always changing. For example, user profile tables kept getting new attributes. Event logs would have columns renamed. Autoload would break every time. I introduced mapping rules in the transformation layer so that if a new field arrived or a name changed, the system could handle it gracefully. That kept bronze to silver flows running smoothly while the product teams made changes upstream.

Safer rollouts with preflight checks including event streams

- To keep compliance dashboards and trading metrics correct, I introduced preflight checks before any DAG could write downstream. First, we checked if DMS arrivals were recent. Second, we checked freshness of critical data like trades, orders, and user activity. If they were stale, the run stopped with a clear error in Airflow.
- In Databricks, I exposed job signals that really mattered: runtime, rows read and written, failure mode, and skew metrics tuned for the kind of trading spikes Gemini sees. This gave the operations team visibility into whether the problem was upstream in DMS, inside Databricks, or downstream in merges.
- I also extended the same principle to event streams. Jobs first checked that topics were alive before they started. If the topic was silent, the job failed fast instead of creating empty tables that still looked like success. This kept downstream teams from being misled.

Enforced masking in shared layers

- Gemini has strict compliance rules. Sensitive fields like customer details and transaction data had to be masked, but engineers sometimes used raw tables in shared notebooks. That opened the door to accidental exposure.
- I enforced a fail-fast mechanism. If someone tried to query a raw table where a masked version existed, the job would fail immediately and point them to the masked view. This kept everyday work in line with compliance requirements and reduced the risk of any data slipping out.

Cutting slow joins back down

- As datasets grew, joins involving trade and order history began creeping into hour-long runtimes. That slowed down surveillance and regulatory reports.
- I scheduled regular housekeeping like clustering and vacuuming on the join keys. I also archived older state that was no longer needed in hot paths but was bloating queries. After that, the heavy joins, especially trade-to-order, went back to business-acceptable runtimes and regulatory reporting stayed on schedule.

Snowflake retirement

- During the transition, old pipelines were still writing into Snowflake while new ones were writing into the lake. That risked creating two versions of the truth, which for a regulated exchange is a nightmare.
- I took control of the cut-over. Old Snowflake jobs were marked for retirement, new work was blocked from writing there, and all builds were redirected into the lake. Source switches were moved into Airflow variables so operations could flip them without touching code. I also created a go or no-go check that validated

freshness, row counts, and dashboard consistency before turning anything off. This gave Gemini a smooth migration where reporting stayed consistent from start to finish.

Mitigating environment drift and state errors with Git and checkpoints

- Many pipelines were still tied to mutable notebooks inside Databricks. Engineers would tweak things in staging, forget to replicate in prod, and suddenly we had drift. Incremental jobs had another problem: they reused the same checkpoints across versions, which caused stale state and even replayed the same window after updates.
- I solved both. I moved all critical notebooks for trades, orders, and balances into Git and made jobs run from fixed commits. That gave us reproducibility and rollbacks. Then I gave each incremental job version its own checkpoint and set up weekly cleanup during low-traffic windows. After that, jobs were predictable, replay problems stopped, and we had a clean audit trail — exactly what a regulated exchange like Gemini requires.

Data Engineer, Fidelity Investments (Brokerage, Clearing and Asset Management Data Platforms), Florida (Hybrid)

Nov 2018 — Jul 2022

- At Fidelity, the systems were powerful but old. The company's core data still came from IBM DB2 mainframes and big warehouse appliances like Teradata and Netezza. These platforms were rock solid and processed millions of trades every night, so regulators trusted them. But they were stiff and slow to change. A schema update or a new feed required long approvals and heavy manual work. Overnight ETL was the norm, because regulators like CAT and FINRA TRACE required certified end-of-day numbers. A few intraday loads were allowed, but the backbone remained end-of-day.
- Around 2018, Fidelity began shifting heavy computation into AWS to scale elastically, and Kubernetes became the official container platform. Before then, there had only been small container pilots. This was the first time containers were rolled out at enterprise scale.
- My team's job was to modernize the parts of the pipeline we owned. That meant fixing skewed Spark jobs, stabilizing compliance feeds, reducing lag in order processing during market bursts, and aligning research pipelines with production so results were consistent.

Portfolio Analytics Engine for Equities and ETFs

- One of the most important systems was the analytics engine for equities and ETFs. It was supposed to give stable analytics across thousands of tickers, but the architecture caused skew and instability. Spark on EMR partitioned data by load date, so big tickers like Apple and Tesla built up massive partitions while small caps zipped through instantly. The imbalance made runtimes unpredictable for downstream users.
- I redesigned the partitioning scheme. By splitting on both symbol and date, the workload was spread evenly and allowed incremental recomputation when needed. Runtimes became balanced, though I had to tune carefully so S3 did not explode with too many small files.
- Once batch skew was fixed, streaming needed its own attention. Intraday ticks came through Kinesis with price, size, and timestamp fields. Dashboards looked healthy if consumers were alive, but that didn't show if shards were moving. Sometimes partitions stalled quietly and stale VWAP or RSI values leaked downstream.
- To prevent that, I built cadence checks to make sure new files landed on time, and throughput checks to compare partition growth against expected trade volumes. These ran in CloudWatch, and if something looked wrong, the job failed early with a clear error. That way, downstream tables were either fresh or they didn't exist at all.
- Vendor feeds introduced constant changes, and this added complexity. New fields such as spread flags would appear, columns like last price would be renamed to trade price, and new routing codes came in. Each change used to break pipelines and force reruns. I solved it with versioned mappers in Git that translated between schema versions. If a new field appeared, it defaulted to null until downstream was ready. Production stayed steady, while analysts tested new volatility indicators, new venue codes, or depth-of-book fields in sandbox mode without breaking live pipelines.

Data Lake for Regulatory and Surveillance Reporting

- Another key responsibility was regulatory and surveillance reporting. Compliance teams needed to monitor trades for best execution, order routing, insider trading patterns, and TRACE submissions. The old approach relied on DB2, Teradata, and Netezza. Each system produced rigid extracts. They were dependable, but every team pulled separately and wasted time reconciling field definitions.
- As part of the AWS program, I helped migrate these feeds into a Lake Formation-backed data lake. This created one consistent order and execution model across teams, so compliance and surveillance staff stopped arguing about whether exec quantity meant the same thing as shares.
- Data checks had always existed, but they were scattered SQL scripts owned by different groups. I folded them into Glue jobs with AWS Deequ. A common one was "orders = executions + cancels." If that check failed, the job stopped and Deequ produced a mismatch report. Errors surfaced early, before regulators ever saw a file.
- Metadata began to travel with every dataset — source, transformations, and validation results. That lineage gave auditors a clear trail. For SEC 605/606, FINRA TRACE, or CCAR audits, they could trace datasets in hours instead of days of spreadsheet reconstruction.

Real-Time Order Processing System

- Reliability in real-time order processing was critical, especially at 9:30 a.m. when the market opened and order flow jumped 5 to 10 times baseline. Our slice of the order path still ran on IBM MQ buses with checkpoint files. MQ guaranteed delivery and sequencing, but during spikes it lagged badly and duplicates slipped in.
- Fidelity had a central Kafka program with massive clusters. I migrated our stream into Kafka Streams. With transactional producers and exactly-once semantics, every trade was processed once and only once. That eliminated both duplicates and data loss, which mattered greatly for reconciliation and compliance.
- Scaling had been tied to CPU and memory, but those metrics didn't reflect order health. I extended Kubernetes autoscaling to look at trading-aware metrics. Consumer lag showed if we were behind, checkpoint clearance showed if state caught up, commit latency showed if writes slowed, and rebalance churn showed if groups were unstable. These signals tied directly to business performance. At the open, when volume spiked, pods scaled automatically.
- Risk checks already existed — things like position limits per account, abnormal price checks against NBBO, and throttles on per-client rates. I embedded them inside Streams jobs so they ran in-line with ingestion. This cut latency and put safeguards directly in the order flow.

Robo-Advisor Research and Recommendations (Fidelity Go)

- Fidelity Go, the robo-advisor launched in 2016, aimed to give clients automated portfolios. The problem was drift. Researchers experimented offline, but production allocations came from separate scripts. Results didn't always match, and reviewers had no clear visibility.
- I co-built a SageMaker pipeline with quants and developers. Researchers ran wide parameter sweeps with different covariance and factor settings, while nightly production ran the same container and commit with locked end-of-day inputs. Drift disappeared, and runs became reproducible.
- The allocation models started with Markowitz mean-variance optimization. I extended them with turnover caps to limit trading churn, transaction cost penalties to avoid expensive trades, and tax-lot logic to minimize short-term gains. These adjustments made portfolios more realistic for client accounts.
- Each nightly run produced a diff of portfolio weights: old versus new, with drivers like volatility shifts, correlation changes, or factor tilts. Before, outputs were black-box. Now investment teams saw the reasons behind changes and could approve quickly.

Time-Series Market Data Store

- Analysts needed intraday snapshots to answer questions fast, like “What was VWAP since the open?” or “How did this sector move in the last 15 minutes?” The existing tools didn’t fit. kdb+ was reserved for trading desks, and Oracle was tuned for overnight reporting. Neither could support quick ad-hoc queries.
- Using Fidelity’s approved architecture pattern, I implemented a team-scoped InfluxDB sidecar. Bulk Go writers handled thousands of ticks per second at open and close. Hot and cold retention balanced cost and performance, and rollups archived older data into S3 but kept it queryable.
- This design matched how analysts actually worked. Instead of waiting on kdb+, they could get sector snapshots in seconds without disturbing production. VWAP and OHLC aggregates were kept fresh through continuous queries, updated incrementally so analysts didn’t have to recompute from scratch. They pulled current metrics instantly, saving both time and compute.

Education

Penn State University
Bachelor of Science in Computational Energy Systems Engineering

Certifications

AWS Certified Data Engineer – Associate (DEA-C01)

AWS Certified Machine Learning – Specialty

Certified Kubernetes Administrator (CKA)

HashiCorp Certified: Terraform Associate

Additional Information

Links: LinkedIn, GitHub
Languages: English, French